

Answer 1: Ten header files contain actual working code: `Stack.hpp`, `SinglyLinkedList.hpp`, `Queue.hpp`, `PriorityQueue.hpp`, `MinHeap.hpp`, `MaxHeap.hpp`, `LinkedList.hpp`, `DoublyLinkedList.hpp`, `Deque.hpp`, and `ArrayList.hpp`. These files have real implementations like constructors that set up the data structures, destructors that clean up memory, and methods that actually do the work of inserting, removing, and organizing data. Only three header files are purely abstract: `AbstractList.hpp`, `TraversableList.hpp`, and `myList.hpp`. These files just define what functions should exist and what the basic structure looks like, but they don't contain any actual working code that does anything.

Answer 2: Different modules work independently because they inherit from different base classes and store their data in completely different ways. For example, `Stack.hpp` and `Queue.hpp` can be changed independently even though they both inherit from `AbstractList.hpp` - `Stack` works like a pile of plates where you only add and remove from the top, while `Queue` works like a line where you add at the back and remove from the front. Similarly, `ArrayList.hpp` and `SinglyLinkedList.hpp` are independent despite both inheriting from `TraversableList.hpp` because `ArrayList` stores elements in a continuous block of memory like an array, while `SinglyLinkedList` connects elements through pointers like a chain. As long as these modules keep their public function names and parameters the same, you can completely rewrite how they work internally without breaking other modules.

Answer 3: The header files show only the essential data members needed for the classes to function, not detailed implementation secrets. While `Stack.hpp` shows its `Node` structure and stack pointer, and `ArrayList.hpp` shows its data array and capacity, these are just the minimum building blocks required for the data structures to exist. The actual clever algorithms for how `push()`, `pop()`, and `sort()` work are hidden inside the method implementations. Different data structures implement the same operations in totally different ways - `Stack` uses a last-in-first-out approach, `Queue` uses first-in-first-out, and `ArrayList` uses dynamic array resizing - but these algorithmic differences are kept private within the method bodies. The only time headers reveal specific implementation details is when the class is designed for a particular approach, like `MinHeap.hpp` and `MaxHeap.hpp` which must show their comparison logic, or `PriorityQueue.hpp` which exposes its heap structure because that's fundamental to what a priority queue is.

Answer 4: Yes, you can use any custom class with these templates, not just basic types like `int` or `double`. The template system works with any class that provides the necessary operations. For example, you could create a `Student` class with comparison operators and use it in `Stack<Student>` `studentStack` or `MinHeap<Student>` `studentHeap`. You can even nest templates inside each other, like `ArrayList<Stack<int>>` for a list of stacks or `Queue<DoublyLinkedList<string>>` for a queue of linked lists. The main requirement is that your custom class must support the operations the template needs - `MinHeap` and `MaxHeap` require comparison operators like `<` or `>`, while general usage needs copy constructors and assignment operators which most classes have automatically. A practical example would be creating a `Point` class for coordinates that implements operator`<` based on distance from origin, allowing `MinHeap<Point>` to automatically order points by how close they are to the center.

Answer 5: No, you cannot create a list with mixed types using the current implementation because C++ templates require one specific type to be chosen when the code is compiled. All elements in `Stack<T>`, `Queue<T>`, or `ArrayList<T>` must be exactly the same type `T`. To store different types together, we need to modify the implementation using two main approaches. The first approach uses object-oriented inheritance by creating a base class like `Element` with specific derived classes like `IntElement` and `StringElement`, then storing smart pointers in something like `ArrayList<std::unique_ptr<Element>>`. The second approach involves creating an entirely new `HeterogeneousAbstractList` class that uses void pointers along with type tracking information to remember what type each stored element actually is. The inheritance approach is generally better because it's safer, follows good programming practices, and prevents many errors, although it uses more memory and requires dynamic allocation. The void pointer approach gives more direct control but sacrifices safety and requires careful manual memory management.

Answer 6: No, you cannot answer yes to the heterogeneous list question without first answering yes to the custom classes question because there's a logical dependency between them. The custom classes question establishes whether templates can work with user-defined classes beyond primitive types like `int` and `double`, which is essential groundwork for understanding template flexibility. If templates could only work with primitive types, then creating heterogeneous lists would be impossible because both proposed solutions require the ability to work with custom classes - either wrapper classes in the inheritance approach or type-tracking structures in the void pointer approach. The affirmative answer to the custom classes question proves that the template system has the flexibility to work with any type that satisfies the required operations, which is the foundation needed to implement the more complex heterogeneous storage solutions. Without this established flexibility, neither the polymorphic approach using base classes nor the void pointer approach using custom type information structures would be feasible within the existing template framework.

Answer 7:

Question 1: One complete module for each new type, no two new types in one module (Score: 16/16) The implementation achieves perfect organization with each header file defining exactly one primary class. `AbstractList.hpp` contains only the `AbstractList` class, `Stack.hpp` defines only the `Stack` class, `Queue.hpp` has only the `Queue` class, and this pattern continues across all twelve modules. While some files like `MinHeap.hpp` and `MaxHeap.hpp` include small helper structures like comparators, these are tightly coupled utility components that directly support the main class rather than independent types. Each class exists in exactly one module, creating clean separation that avoids the confusion and maintenance problems that would arise if classes were scattered across multiple files or if unrelated types were mixed together in single files.

Question 2: A single header for each new type (Score: 52/52) Every type in the implementation has its own dedicated header file, achieving complete separation at the file level. The twelve distinct types each reside in their respective header files without any type definitions being duplicated across multiple files or any single type being split across multiple headers. The `myList.hpp` file serves purely as a convenience aggregation header that includes all other headers for easy importing but does not

define any types itself, preserving the one-type-per-header principle throughout the entire codebase.

Question 3: No imperative code in headers (Score: 6/26) Only three header files successfully avoid containing executable implementation code: `AbstractList.hpp`, `TraversableList.hpp`, and `myList.hpp`. `AbstractList.hpp` contains primarily pure virtual function declarations with only simple inline accessor methods, `TraversableList.hpp` consists entirely of abstract function declarations and basic member variable definitions, and `myList.hpp` contains only include directives. The remaining nine headers all contain substantial implementation code including constructors that initialize data structures, destructors that manage memory cleanup, and complex algorithms for insertion, deletion, sorting, and traversal operations. This implementation code is necessary in C++ template headers for proper instantiation and optimal performance, but it does expose implementation details.

Question 4: Minimal classes needing change if base storage structure changes (Score: 4/4) The implementation demonstrates excellent object-oriented design principles by placing all concrete implementations at the most derived level possible, ensuring that changes to `AbstractList`'s storage approach would require zero classes to be modified. The abstract base classes define only interfaces and basic member variables without depending on any specific storage implementation details, while each concrete class implements its own completely independent storage mechanism. Since `AbstractList` itself is purely abstract and doesn't dictate any particular storage strategy, concrete classes operate in complete isolation - `Stack` uses its own linked node system, `ArrayList` manages its own dynamic array, and `Queue` maintains its own linked structure. Even if we wanted to change the internal storage implementation of a specific derived class, only that individual module would need modification while all other classes remain completely unaffected, showcasing the power of proper abstraction in minimizing cross-class dependencies.

We can achieve a full score if we make the changes stated above.